

Patristic Library Assistant

Build-Ready Bounded Multi-Agent Architecture Specification

Version 3 - developer handoff draft

Scope: query-time orchestration, workflow-time orchestration, schemas, endpoints, module structure, sequencing, and operational guardrails.

Document Type	Engineering architecture specification
Primary Goal	Implement bounded multi-agent orchestration without violating closed-corpus discipline
Intended Reader	Full-stack engineer, technical co-founder, contract developer
Stack Assumption	FastAPI, Next.js, PostgreSQL, Qdrant, Redis, background queue, Claude API
Status	Build-ready design; implementation choices still require final naming and migration review

1. System objective

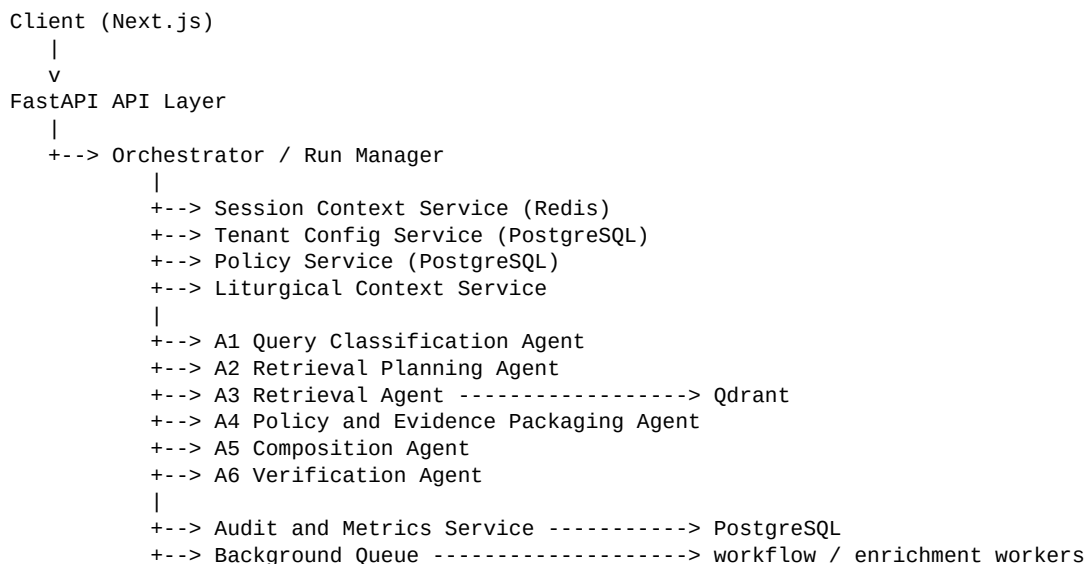
Design a **bounded multi-agent architecture** for a closed-corpus theological AI product. The system should gain the advantages of specialization and orchestration while preserving the product's non-negotiable trust constraints: approved evidence only, tenant isolation, auditable citations, explicit confidence handling, and policy-aware output.

Non-negotiable design constraints

- No agent may rely on external knowledge when producing answers or artifacts.
- Only approved tenant-visible evidence may be used for user-facing outputs.
- Governance rules and policy inheritance must be applied before final composition.
- Every final answer or generated artifact must pass a verification stage.
- All agent steps must be observable, logged, and reproducible enough for debugging.

2. Architecture overview

The architecture uses a **single orchestrator** with **narrow specialist agents**. Agents do not operate as autonomous researchers. They act as bounded workers with fixed input/output contracts.



Why bounded agents instead of free-roaming agents

- Deterministic control over evidence boundaries.
- Lower hallucination risk than open-ended agent collaboration.
- Easier latency budgeting for query-time responses.
- Much better observability and incident debugging.
- A clear procurement and trust story for seminaries, dioceses, and monasteries.

3. Agent definitions

Agent	Primary responsibility	Allowed inputs	Hard prohibitions
A1 Query Classification	Classify query type, sensitivity, answer mode, user query, session context, tenant config, request id, session id	Classified query, session context, tenant config, request id, session id	No configuration updates; no generation
A2 Retrieval Planning	Build retrieval template, filters, boosts, k value, liturgical context, policy, role, tier	Classified query, tenant config, liturgical context, policy, role, tier	No invocation; no test; no final inclusion decision
A3 Retrieval	Execute vector search and return scored chunks	Retrieval plan	No governance filtering; no composition
A4 Policy and Evidence Application	Apply governance, attestation, citation health, retrieve chunks, policy, package, role, tier	Retrieved chunks, policy, package, role, tier	No re-attestation; no re-format; no answer writing
A5 Composition	Draft answer or artifact from evidence packet	Evidence packet, prompt rules, answer mode	No external knowledge; no missing-evidence invocation
A6 Verification	Validate citations, schema completeness, and draft output	Draft output, evidence packet, classified query	No duplication; may only approve, reject, or triage

4. Runtime sequence diagrams

4.1 Query-time sequence

```
User -> API /query
API -> Orchestrator: create run_id
Orchestrator -> A1 Classifier: classify(query, session, tenant)
A1 -> Orchestrator: classified_query
Orchestrator -> A2 Planner: build_plan(classified_query, tenant, liturgical_context)
A2 -> Orchestrator: retrieval_plan
Orchestrator -> A3 Retrieval: execute(plan)
A3 -> Orchestrator: scored_chunks
Orchestrator -> A4 Evidence: package(chunks, policy, attestation, role)
A4 -> Orchestrator: evidence_packet + preliminary_tier
Orchestrator -> A5 Compose: draft(evidence_packet, answer_mode)
A5 -> Orchestrator: response_draft
Orchestrator -> A6 Verify: validate(draft, evidence_packet, classified_query)
A6 -> Orchestrator: verified_response
Orchestrator -> Audit Service: persist run logs + metrics
Orchestrator -> API: final payload
API -> User: answer + citations + metadata
```

4.2 Workflow-time sequence

```
User/Admin -> API /workflows/run
API -> Orchestrator: create workflow_run
Orchestrator -> Queue: enqueue(workflow_type, inputs)
Worker -> A1 Classifier: classify as workflow variant
Worker -> A2 Planner: broad retrieval plan
Worker -> A3 Retrieval: collect relevant chunks
Worker -> A4 Evidence: package and group evidence
Worker -> A5 Compose: build draft artifact
Worker -> A6 Verify: validate artifact sections and citations
Worker -> Export Service: generate PDF/DOCX/JSON
Worker -> DB: persist output artifacts
Admin -> API /workflows/{id}/approve
API -> Publish/Return final artifact
```

5. Core data contracts

5.1 ClassifiedQuery

```
type ClassifiedQuery = {
  tenantId: string
  userId: string
  rawQuery: string
}
```

```

reframedQuery?: string
queryType: "question" | "workflow" | "admin" | "scholarly"
sensitivity: "normal" | "sensitive" | "political" | "medical" | "pastoral"
answerMode:
  | "direct_citation"
  | "consensus"
  | "historical_development"
  | "institutional_policy"
  | "scholarly_dispute"
needsLiturgicalBoost: boolean
needsPolicyMode: boolean
shouldReframe: boolean
}

```

5.2 RetrievalPlan

```

type RetrievalPlan = {
  semanticQuery: string
  retrievalTemplate: "default" | "consensus" | "historical" | "policy" | "scholarly"
  filters: {
    tenantId: string
    approved: true
    language?: string[]
    sourceScope?: Array<"tenant" | "starter_corpus">
    categories?: string[]
    fatherCited?: string[]
  }
  boosts: {
    liturgicalPeriod?: string[]
    depthLevel?: string[]
    chronologyBias?: "early" | "balanced" | "late"
  }
  k: number
}

```

5.3 EvidencePacket

```

type EvidencePacket = {
  allowedChunks: Chunk[]
  suppressedChunks: Chunk[]
  scholarlyOnlyChunks: Chunk[]
  attestationSummary: CitationAttestation[]
  citationHealthSummary: CitationHealth[]
  governanceExplanation: {
    policyNode: string
    appliedRules: string[]
    roleView: "member" | "admin" | "scholar"
  }
  confidenceTier: "GREEN" | "YELLOW" | "SENSITIVE" | "RED"
  coverageStatus: "sufficient" | "limited" | "insufficient"
}

```

5.4 VerifiedResponse

```

type VerifiedResponse = {
  valid: boolean
  responseAction: "return_to_user" | "regenerate" | "fallback"
  finalTier: "GREEN" | "YELLOW" | "SENSITIVE" | "RED"
  issues: string[]
  payload: Record<string, unknown>
}

```

6. Storage model and ERD

Qdrant remains the semantic retrieval layer. PostgreSQL is the system of record for configuration, policies, audit, workflows, and review state. Redis stores ephemeral session state and optionally short-lived orchestration locks.

6.1 Core relational tables

```
tenants(id, name, language, starter_corpus_enabled, model_preference, monthly_query_limit, ...)
users(id, tenant_id, role, ...)
query_logs(id, tenant_id, user_id, query_text, answer_mode, tier, created_at, ...)
agent_runs(id, run_type, tenant_id, user_id, parent_run_id, step_name, model_name,
            input_hash, output_hash, status, latency_ms, token_in, token_out, created_at)
workflow_runs(id, tenant_id, workflow_type, initiated_by, status, inputs, outputs,
              approved_by, created_at)
workflow_steps(id, run_id, step_type, status, started_at, finished_at, metadata)
agent_review_events(id, tenant_id, run_id, severity, issue_type, details, resolved_by, created_at)

policy_nodes(id, tenant_id, parent_node_id, node_type, name)
policy_rules(id, policy_node_id, rule_key, rule_value, precedence, active_from, active_to)
corpus_scope_rules(id, policy_node_id, source_id, visibility_mode)

source_attestations(id, source_id, source_hash, extraction_method, transcription_method,
                    transcription_confidence, translation_status, original_language, created_at)
chunk_attestations(chunk_id, attestation_id, quote_integrity, approved_by, approved_at,
                  manual_review_required, review_notes)

citation_candidates(id, source_chunk_id, extracted_quote, cited_father, cited_work,
                  cited_location_text, normalized_quote)
citation_matches(candidate_id, canonical_chunk_id, lexical_score, embedding_score,
                combined_score, classification, reviewer_id, reviewer_note)
```

6.2 ERD sketch

```
[tenants] 1---n [users]
[tenants] 1---n [query_logs]
[tenants] 1---n [agent_runs]
[workflow_runs] 1---n [workflow_steps]
[policy_nodes] 1---n [policy_rules]
[policy_nodes] 1---n [corpus_scope_rules]
[source_attestations] 1---n [chunk_attestations]
[citation_candidates] 1---n [citation_matches]

Qdrant payload (per chunk):
chunk_id, tenant_id, source_id, approved, source_title, source_url, source_type,
timestamp_start, timestamp_end, father_cited[], categories[], depth_level,
language, liturgical_period[], embedding_model_version, related_chunks[], builds_on[],
content_text, ingested_at
```

7. API surface

Endpoint	Purpose	Caller	Notes
POST /api/v1/query	Synchronous bounded query execution	Member / Admin	Returns answer, citations, cross-reference panel, verification
GET /api/v1/query/history	Read prior sessions or queries	Member	Filtered to current tenant and user permissions
POST /api/v1/workflows/run	Start bounded workflow	Admin / Staff	Returns workflow_run_id and queued status
GET /api/v1/workflows/{id}	Workflow status and artifacts	Admin / Staff	Includes step log and validation result
POST /api/v1/workflows/{id}/approve	Approve generated artifact	Admin	Required for public/published artifacts
GET /api/v1/runs/{run_id}	Debug run trace	Admin / Developer	Used for incident debugging and QA
GET /api/v1/policies/tree	Governance tree inspection	Admin	Supports simulation UI

7.1 Example response shape for /query

```
{
  "tier": "GREEN",
  "answer_mode": "consensus",
  "answer": {
    "agreement_points": [...],
    "divergence_points": [...],
    "conciliar_resolution": [...]
  },
  "citations": [...],
  "cross_reference_panel": [...],
  "attestation": [...],
  "governance_explanation": {...},
  "verification": {
    "valid": true,
    "issues": []
  }
}
```

8. Recommended module structure

```
backend/
  app/
    api/
      v1/
        query.py
        workflows.py
        runs.py
        policies.py
        ingest.py
    core/
      config.py
      logging.py
      security.py
      prompts/
        base_prompt.py
        mode_prompts.py
    domain/
      agents/
        classifier.py
        retrieval_planner.py
        retrieval.py
        evidence_packager.py
        composer.py
        verifier.py
      services/
        qdrant_service.py
        policy_service.py
        attestation_service.py
        citation_service.py
        liturgical_service.py
        session_service.py
        export_service.py
        audit_service.py
      workflows/
        base_workflow.py
        study_packet.py
        content_brief.py
        bishop_brief.py
      models/
        query_models.py
        workflow_models.py
        policy_models.py
        attestation_models.py
```

```

repositories/
  tenant_repo.py
  query_repo.py
  policy_repo.py
  workflow_repo.py
  audit_repo.py
workers/
  queue.py
  tasks/
    workflow_tasks.py
    citation_tasks.py
    clustering_tasks.py
    alignment_tasks.py
tests/
  integration/
  unit/
  safety/
frontend/
  app/
    chat/
    admin/
    workflows/
    runs/
  components/
    answer/
    citations/
    governance/
    workflow/
    trust/

```

8.1 Separation rules

- Agents contain orchestration-time reasoning and transformation logic.
- Services own infrastructure access, such as Qdrant, PostgreSQL, Redis, and external APIs.
- Repositories own database read/write patterns.
- Workflow modules compose existing agents and services; they do not duplicate prompt logic.
- Prompt text should be versioned and mode-aware rather than embedded ad hoc in route handlers.

9. Verification and failure behavior

Rule	Failure if violated
Every paragraph or section has source support	Fallback or regeneration
No suppressed chunk is cited	Hard failure
Sensitive-mode answer contains no personal advice wording	Hard failure
Answer-mode required sections are present	Regeneration or fallback
Final tier is consistent with evidence coverage	Tier downgrade and logged warning
Institutional-policy outputs disclose local scope	Hard failure

Failure strategy:

1. Validate draft
2. If recoverable, allow one controlled regeneration
3. If still invalid, return bounded fallback (YELLOW, SENSITIVE, or RED)

4. Persist review event for admin or QA inspection

10. Workflow catalog

Workflow	Primary output	Inline or async	Approval required
Study packet	PDF/DOCX packet grouped by Father/topic	Async	Yes
Content brief	Editorial brief for unanswered query clusters	Async	No for draft; yes for distribution
Bishop briefing	Institutional summary with policy-safe evidence	Async	Yes
Lecture-to-guide conversion	Study guide / FAQ draft	Async	Yes

11. Phased implementation plan

Phase	Build	Do not build yet
Phase 1	A1 classifier, A3 retrieval, A5 composition, A6 verification; basic templating engine, async workflow catalog, citation resolver	Policy inheritance, answer mode
Phase 2	A2 planner, A4 evidence packager, liturgical boosts, answer fallback, attribution package	Full distribution package, first review console
Phase 3	Async workflows, export service, approval checkpoints, workflow status	Cross-tenant network activation
Phase 4	Background enrichment: citation resolution, clustering, alignment, deep agent collaboration	Intelligence augmentation

12. Operational requirements and test strategy

12.1 Required observability

- Per-agent latency and token usage.
- Run trace retrieval by run_id.
- Fallback rate and regeneration rate.
- Validation failure categories.
- Tier distribution by tenant and answer mode.

12.2 Must-have test suites

- Closed-corpus safety tests.
- Sensitive query reframing tests.
- Policy inheritance and scholarly-only visibility tests.
- Mode-specific schema completeness tests.
- Workflow insufficient-coverage tests.
- Caching and repeated-query determinism tests.

13. Explicit anti-patterns

- Do not let multiple agents independently retrieve and merge evidence.
- Do not let the composition agent see suppressed evidence unless the mode explicitly allows scholarly context.
- Do not generate public-facing artifacts without approval checkpoints.
- Do not embed prompt logic directly inside route handlers.
- Do not add open-web browsing to answer-time agents for this product category.
- Do not use agent debate as a substitute for evidence verification.

14. Implementation directive

Build an orchestrated agent system where **specialization exists inside hard constraints**. Use FastAPI as the orchestrator, Qdrant as the only semantic retrieval backend, PostgreSQL as the system of record, Redis for short-lived state, and a background queue for all non-interactive jobs. The agents exist to improve retrieval planning, governance enforcement, evidence packaging, and output verification - not to simulate unrestricted intelligence.

Deliverable target for first implementation: a query endpoint that is measurably safer and more structured than a generic single-prompt RAG path, with enough observability that an engineer can diagnose every failed answer path.